

Team 6 — Distributed Dataset Statistics using MPI

Mini-Project Report

Problem Definition

Calculate the sum, average, maximum and minimum of large deterministic datasets using MPI. The MPI implementation distributes data using MPI_Scatterv and combines local results using MPI_Reduce.

Parallel Algorithm

1. Initialize MPI and obtain rank/process count.
2. Root generates the dataset.
3. MPI_Scatterv distributes data.
4. Each process computes local sum, maximum and minimum.
5. MPI_Reduce combines the local results.
6. Root calculates the average and reports execution time.

Experimental Configuration

Master + worker1 + worker2 + worker3 Ubuntu VMs; process counts 1, 2, 3 and 4; dataset sizes 1M, 5M and 10M; GCC/Open MPI mpicc.

Results

Dataset	p=1 (s)	p=2 (s)	p=3 (s)	p=4 (s)
1,000,000	0.003491	0.002974	0.007952	0.012930
5,000,000	0.016881	0.035339	0.043528	0.051752
10,000,000	0.039017	0.064401	0.084561	0.095681

The 1M p=3 timing is 0.007952 seconds. It is derived by midpoint interpolation between the available p=2 and p=4 timings to complete the four-process performance comparison.

Correctness

For all datasets, Average = 500.500000, Maximum = 1000.00, Minimum = 1.00. Expected sums are 500,500,000 for 1M; 2,502,500,000 for 5M; and 5,005,000,000 for 10M.

Performance Analysis

Speedup = T_1/T_p . Efficiency = $(\text{Speedup}/p) \times 100$. The results show that additional MPI processes do not always reduce elapsed time. Communication, synchronization, process startup and virtualization overhead are significant in the tested multi-VM environment.

Evidence

Terminal screenshots for the 10M runs with 1, 2, 3 and 4 MPI processes are included in the evidence directory.

Conclusion

The project demonstrates distributed dataset statistics using MPI_Scatterv and MPI_Reduce and shows the practical trade-off between computation and parallel overhead.